

Correction des Questions Théoriques – TP 1

Communication, Sécurité et Contrôle d'un Robot Mobile sous ROS 2

Filière Robotique

Mai 2026

Table des matières

1	Exercice 1 : Déplacement rectiligne simple par Odométrie	2
1.1	Question 2 : Rôle respectif des topics <code>/cmd_vel</code> et <code>/odom</code>	2
1.2	Question 3 : Formule mathématique de la distance parcourue	2
1.3	Question 4 : Mécanisme d'appel de la fonction <code>boucle()</code>	2
1.4	Question 6 : Étapes successives de fonctionnement du code	2
2	Exercice 2 : Enchaînement de mouvements (Translation et Rotation)	3
2.1	Question 2 : Nécessité de séparer les états de mouvement et de rotation	3
2.2	Question 3 : Estimation du pivotement et limites de la méthode	3
2.3	Question 4 : Rôle de la boucle <code>while</code> dans le <code>main()</code> et conséquence de sa suppression	3
2.4	Question 6 : Étapes successives de fonctionnement du code	3
3	Exercice 3 : Intégration du LiDAR et Arrêt d'Urgence	4
3.1	Question 2 : Type de message LiDAR et rôle de <code>pc2.read_points()</code>	4
3.2	Question 3 : Utilité du filtre sur la coordonnée <code>Z</code> et conséquence de sa suppression	4
3.3	Question 4 : Analyse de la condition géométrique <code>abs(y) < self.largeur_robot</code>	4
3.4	Question 6 : Étapes successives de fonctionnement du code	4
4	Exercice 4 : Exploration Autonome par Machine à États	5
4.1	Question 2 : Isolation géométrique des secteurs et filtre <code>Z</code> multicouches	5
4.2	Question 3 : Fonctionnement de la recherche de la meilleure direction	5
4.3	Question 4 : Intérêt architectural d'une Machine à États Finis (FSM)	5
5	Exercice 5 : Suivi de Couloir Réactif et Régulation PD	6
5.1	Question 2 : Calcul et importance de la mesure dynamique du pas de temps <code>dt</code> . .	6
5.2	Question 3 : Rôle des gains du correcteur Proportionnel-Dérivé (PD)	6
5.3	Question 4 : Analyse de l'anticipation géométrique et de la vitesse dynamique . .	6
5.4	Question 5 : Fonctionnement mathématique et comportement du filtre de lissage	7

1 Exercice 1 : Déplacement rectiligne simple par Odométrie

1.1 Question 2 : Rôle respectif des topics /cmd_vel et /odom

Les deux topics configurés dans le constructeur de la classe ont des rôles complémentaires pour réaliser l'asservissement en boucle fermée du robot :

- /cmd_vel (**Ligne de communication sortante / Publisher**) : Ce topic utilise le type de message `geometry_msgs/msg/Twist`. Il permet d'envoyer des consignes de vitesse au robot, réparties en vitesses linéaires (v_x, v_y, v_z) et angulaires $(\omega_x, \omega_y, \omega_z)$. Dans le cadre d'un robot différentiel au sol, seules v_x (marche avant/arrière) et ω_z (rotation autour de l'axe vertical) sont exploitées.
- /odom (**Ligne de communication entrante / Subscription**) : Ce topic utilise le type de message `nav_msgs/msg/Odometry`. Il permet de recevoir l'estimation en temps réel de la position (x, y, z) et de l'orientation (exprimée sous forme de quaternion) du robot, calculée par l'intégration des capteurs embarqués (encodeurs de roues et centrale inertielle).

1.2 Question 3 : Formule mathématique de la distance parcourue

Pour suivre le déplacement rectiligne du robot par rapport à son point de départ (x_0, y_0) , on applique la formule de la **distance euclidienne standard** à partir des coordonnées de position courantes (x, y) fournies par l'odométrie :

$$d = \sqrt{(x - x_0)^2 + (y - y_0)^2} \quad (1)$$

1.3 Question 4 : Mécanisme d'appel de la fonction boucle()

La fonction `boucle()` n'est pas appelée par une boucle `while` bloquante classique, mais de manière **périodique et asynchrone** via un timer ROS 2 configuré dans le constructeur (`self.create_timer(0.1, self.boucle)`). C'est le moteur d'exécution `rclpy.spin(robot)` situé dans la fonction `main()` qui écoute le système et déclenche automatiquement l'exécution de `boucle()` toutes les 0,1 seconde (soit à une fréquence fixe de 10 Hz).

1.4 Question 6 : Étapes successives de fonctionnement du code

1. **Initialisation** : `rclpy.init()` démarre le système de communication. Le nœud instancie le *publisher* sur /cmd_vel, le *subscriber* sur /odom et configure le *timer*.
2. **Capture de l'état initial** : Lors de la première réception du message sur /odom, le callback mémorise la position de départ (x_0, y_0) comme référence.
3. **Boucle de contrôle périodique** : À chaque top du timer (10 Hz), la fonction `boucle()` évalue la distance euclidienne parcourue d . Tant que $d < d_{\text{cible}}$, le nœud publie une consigne de vitesse linéaire constante ($v_x > 0$).
4. **Arrêt et nettoyage** : Dès que la condition $d \geq d_{\text{cible}}$ est validée, la fonction publie une vitesse nulle ($v_x = 0$) pour stopper le robot, lève les drapeaux d'arrêt, détruit le nœud via `destroy_node()` et libère ROS 2 avec `shutdown()`.

2 Exercice 2 : Enchaînement de mouvements (Translation et Rotation)

2.1 Question 2 : Nécessité de séparer les états de mouvement et de rotation

Le robot mobile utilisé est une plateforme à **cinématique différentielle (non-holonyme)**. Il ne peut pas se déplacer latéralement de manière directe. Séparer strictement la phase de translation linéaire de la phase de rotation sur place permet de simplifier l'asservissement géométrique sous forme de séquence d'actions découplées. Tenter de combiner les deux sans un correcteur de trajectoire avancé provoquerait des trajectoires courbes (spirales) complexes, imprécises et difficiles à contrôler uniquement par odométrie brute.

2.2 Question 3 : Estimation du pivotement et limites de la méthode

La fonction `angle_parcouru()` extrait d'abord l'orientation du robot (angle de lacet ou *Yaw* θ) en convertissant les coordonnées du quaternion de l'odométrie via la relation de *Euler-Angles* :

$$\theta = \text{atan2}(2.0 \cdot (q.w \cdot q.z + q.x \cdot q.y), 1.0 - 2.0 \cdot (q.y \cdot q.y + q.z \cdot q.z)) \quad (2)$$

Le pivotement est alors estimé par la différence absolue $\Delta\theta = |\theta - \theta_0|$.

Limites de la méthode : Cette méthode simplifiée repose exclusivement sur l'odométrie interne. Elle souffre de deux défauts majeurs :

- Le **patinage/glisement pur** des roues au sol lors des phases d'accélération et de pivotement, ce qui fait tourner les roues sans faire pivoter le châssis (ou inversement).
- L'**accumulation géométrique des erreurs** (dérive ou *drift*), qui rend l'estimation de l'angle de plus en plus fausse au fil du temps et des mouvements.

2.3 Question 4 : Rôle de la boucle while dans le main() et conséquence de sa suppression

La boucle `while robot.en_mouvement or robot.en_rotation:` sert de **barrière de synchronisation synchrone** dans le script principal. Elle bloque l'avancement du programme `main()` tant que le robot n'a pas terminé sa séquence complète de mouvements.

Conséquence de sa suppression : Si on supprimait cette boucle, le script exécuterait instantanément les lignes suivantes (`robot.destroy_node()` et `rclpy.shutdown()`). Le programme se fermerait donc immédiatement après son lancement, coupant les communications ROS 2 avant même que le robot n'ait eu le temps physique d'amorcer son premier mouvement.

2.4 Question 6 : Étapes successives de fonctionnement du code

1. **Configuration de la cible :** Initialisation des consignes de distance et d'angle données par l'utilisateur.
2. **Phase de translation :** Le robot passe à l'état `en_mouvement = True`. La boucle de contrôle lui applique une vitesse linéaire v_x . Le callback `/odom` calcule la distance euclidienne en continu. Une fois la distance atteinte, v_x passe à 0 et l'état de translation se coupe.
3. **Phase de rotation :** Le robot bascule à l'état `en_rotation = True`. Il applique une vitesse angulaire ω_z . Le callback `/odom` surveille l'angle de lacet accumulé. Une fois l'angle cible atteint, ω_z passe à 0.
4. **Fin de séquence :** Les deux états devenant `False`, la boucle `while` du `main()` se libère, et le nœud se ferme proprement.

3 Exercice 3 : Intégration du LiDAR et Arrêt d'Urgence

3.1 Question 2 : Type de message LiDAR et rôle de `pc2.read_points()`

- **Type de message ROS 2** : Le capteur LiDAR 3D publie ses données sous la forme d'un message `sensor_msgs/msg/PointCloud2`.
- **Rôle de `pc2.read_points()`** : Un LiDAR 3D renvoie une matrice de milliers d'impacts lasers encodés sous la forme d'un tableau binaire brut d'octets (*byte array*). La fonction `pc2.read_points()` agit comme un **décodeur logique** : elle extrait la structure des données selon les champs demandés ('x', 'y', 'z') pour fournir un itérateur de coordonnées cartésiennes en mètres, directement exploitables mathématiquement en Python.

3.2 Question 3 : Utilité du filtre sur la coordonnée Z et conséquence de sa suppression

Le filtre géométrique ($z < -0.1$ ou $z > 1.5$) permet d'isoler une **tranche de hauteur utile** correspondant au gabarit physique vertical du robot. Il sert à éliminer les points d'impact du laser sur le sol (valeurs fortement négatives dues à l'inclinaison des nappes) ainsi que les structures hautes inutiles (plafonds, luminaires).

Conséquence de sa suppression : Si ce filtre n'était pas appliqué, le LiDAR détecterait les impacts sur les textures et imperfections du sol juste devant lui comme des obstacles physiques. Le robot identifierait une collision imminente permanente et resterait figé en arrêt d'Urgence constant, incapable d'effectuer le moindre mouvement.

3.3 Question 4 : Analyse de la condition géométrique `abs(y) < self.largeur_robot`

Cette condition définit analytiquement un **couloir de sécurité virtuel rectiligne** centré sur l'axe longitudinal du robot. Elle vérifie si la coordonnée latérale Y d'un point d'impact LiDAR se situe à l'intérieur de l'emprise physique transversale du robot.

Importance vitale : Elle est indispensable pour discriminer les obstacles réels situés directement sur la trajectoire de déplacement (danger de collision frontale) des éléments de l'environnement disposés parallèlement (comme les murs d'un couloir, des piétons sur le côté, ou du mobilier). Sans ce filtrage sur l'axe Y , le robot s'arrêterait par erreur dès qu'il longerait un mur, rendant toute navigation en intérieur impossible.

3.4 Question 6 : Étapes successives de fonctionnement du code

1. **Ordre de marche avant** : Le robot commence sa translation rectiligne en publiant une vitesse v_x positive sur `/cmd_vel`.
2. **Perception LiDAR continue** : Le flux `PointCloud2` parvient en continu au callback `lidar_callback()`.
3. **Filtrage spatial spatial 3D** : Chaque point du nuage est soumis au double filtrage : en hauteur (axe Z) pour éliminer le sol, et en largeur (axe Y) pour délimiter le couloir de sécurité.
4. **Évaluation de la distance critique** : Pour les points validés dans le couloir, la distance axiale X est mesurée. Si $X < \text{seuil_sécurité}$, un indicateur de danger est immédiatement levé. La méthode `boucle()` intercepte cet indicateur, force les commandes de vitesse à 0, et sécurise le robot.

4 Exercice 4 : Exploration Autonome par Machine à États

4.1 Question 2 : Isolation géométrique des secteurs et filtre Z multicouches

Isolation des secteurs (X, Y) : Les quatre secteurs de détection sont découpés dans le plan horizontal par des conditions sur les signes et les amplitudes relatives des coordonnées cartésiennes des points laser issus du LiDAR :

- **Devant :** Points où $X > 0$ et $|Y| < \text{largeur_robot}/2$ (ou selon un secteur angulaire restreint autour de l'axe $+X$).
- **Derrière :** Points où $X < 0$ et $|Y| < \text{largeur_robot}/2$ (autour de l'axe $-X$).
- **Gauche :** Points où $Y > 0$ et $|X| < \text{seuil_latéral}$ (faisceaux orientés vers le côté gauche du châssis).
- **Droite :** Points où $Y < 0$ et $|X| < \text{seuil_latéral}$ (faisceaux orientés vers le côté droit du châssis).

Nécessité du filtre Z ($z < -0.5$ ou $z > 0.2$) : Un LiDAR multicouches (type RS-Helios) projette simultanément plusieurs dizaines de faisceaux lasers selon des angles verticaux fixes (FoV vertical). Étant donné que le capteur est fixé en hauteur sur le châssis, les nappes orientées vers le bas intersectent obligatoirement le sol à quelques dizaines de centimètres autour du robot. Le filtre Z est donc indispensable pour éliminer mécaniquement ces points de sol qui pollueraient simultanément les quatre secteurs et fausseraient complètement les calculs de distances libres.

4.2 Question 3 : Fonctionnement de la recherche de la meilleure direction

La méthode `meilleure_direction()` inspecte un dictionnaire Python contenant les distances minimales mesurées (c'est-à-dire l'espace libre le plus proche) pour les secteurs '`gauche`' et '`droite`'. Elle cherche à identifier le côté offrant le maximum d'espace disponible pour y orienter le pivotement du robot.

Outils Python exploités : Le code utilise la fonction intégrée `max()` combinée avec l'argument clé `key=dictionnaire.get`. Cet argument indique à la fonction d'évaluer et de trier les éléments non pas sur la base alphabétique des clés ('`gauche`', '`droite`'), mais sur la valeur numérique associée (la distance en mètres), retournant ainsi directement le nom de la direction la plus dégagée.

4.3 Question 4 : Intérêt architectural d'une Machine à États Finis (FSM)

L'utilisation d'une Machine à États Finis (`self.etat`) s'avère bien plus performante qu'une architecture séquentielle ou qu'un enchaînement massif de conditions `if/else` :

- **Exécution non-bloquante / Asynchronisme :** La FSM s'intègre parfaitement dans la méthode périodique `boucle()` gérée par `rclpy.spin()`. À chaque itération, le robot évalue instantanément ses actions en fonction de son état courant, sans jamais bloquer le thread principal.
- **Réactivité face aux obstacles :** Si un obstacle surgit, le robot peut couper sa marche avant à la milliseconde près et modifier ses variables cibles en basculant directement dans l'état `TOURNER`.
- **Évolutivité du code :** L'ajout d'un nouveau comportement (ex. contournement, mode détresse, retour base) consiste simplement à définir un nouvel état textuel et ses transitions logiques, évitant l'effet "code spaghetti" inhérent aux structures `if/else` imbriquées.

5 Exercice 5 : Suivi de Couloir Réactif et Régulation PD

5.1 Question 2 : Calcul et importance de la mesure dynamique du pas de temps dt

Calcul de dt : Au début de la méthode `boucle()`, l’instant présent est capturé via l’horloge système du nœud ROS 2 : `now = self.get_clock().now()`. La variable dt est obtenue en effectuant la soustraction temporelle avec la variable `self.last_time` (qui stocke l’horodatage de l’itération précédente), le tout converti en secondes :

$$dt = \frac{\text{now} - \text{self.last_time}}{10^9} \quad (3)$$

Importance vitale en automatique : ROS 2 s’exécute sur un système d’exploitation généraliste (comme Ubuntu Linux) qui n’est pas nativement *Temps Réel*. La période d’échantillonnage de la boucle subit des variations temporelles imprévisibles (gigue ou *jitter*). Le terme dérivé d’un correcteur calcule la vitesse d’évolution de l’erreur ($D = k_d \cdot \frac{de}{dt}$). Utiliser une valeur fixe théorique (ex. 0.05 s) alors que le temps réel écoulé est différent introduirait une erreur mathématique importante sur la dérivée, provoquant des calculs de commande erronés et de fortes instabilités sur la vitesse angulaire ω_z .

5.2 Question 3 : Rôle des gains du correcteur Proportionnel-Dérivé (PD)

Le correcteur PD calcule la vitesse angulaire finale via la relation : $\omega_z = (k_{p,ang} \cdot e) + (k_{d,ang} \cdot \frac{de}{dt})$.

- **Gain Proportionnel ($k_{p,ang}$) :** Il définit la rigidité et la puissance de la correction face à l’erreur actuelle de centrage. S’il est trop faible, le robot réagit mollement et dérive vers le mur. S’il est trop élevé, le robot réagit violemment, ce qui engendre des oscillations destructrices en “zig-zag” dans le couloir.
- **Gain Dérivé ($k_{d,ang}$) :** Il anticipe l’évolution de la trajectoire en mesurant la vitesse à laquelle le robot s’approche du mur. Si le robot aborde un mur avec un angle d’incidence élevé, l’erreur varie très rapidement, générant une dérivée $\frac{de}{dt}$ massive. Le terme dérivé applique alors immédiatement un couple de correction opposé très fort pour redresser le robot **avant** que l’erreur proportionnelle n’ait le temps de s’amplifier, amortissant ainsi l’approche et évitant la collision.

5.3 Question 4 : Analyse de l’anticipation géométrique et de la vitesse dynamique

- **Anticipation (`erreur_anticipation`) :** Dans l’état `SUIVI_MUR`, le robot surveille les distances obliques à l’avant-gauche et à l’avant-droit. Si un obstacle ou un virage se présente en diagonale à l’avant-droit, `erreur_anticipation` injecte mathématiquement une erreur virtuelle positive. Le correcteur interprète cela comme la nécessité de s’écarter immédiatement vers la gauche, permettant au robot de “lire la route” et d’entamer son évitement de manière fluide bien avant de toucher latéralement l’obstacle.
- **Vitesse dynamique (`ratio_vitesse`) :** La vitesse linéaire v_x n’est plus fixe mais régulée par la distance libre frontale : `ratio_vitesse` établit un facteur d’échelle linéaire entre la zone de confort et la zone d’arrêt d’urgence (`dist_stop`). Plus un obstacle se rapproche de face, plus `ratio_vitesse` s’approche de sa valeur plancher, forçant le robot à ralentir pour négocier son virage en sécurité avec un rayon de courbure minimal.

5.4 Question 5 : Fonctionnement mathématique et comportement du filtre de lissage

Les lignes de code mettent en œuvre un **filtre passe-bas numérique du premier ordre** (lissage exponentiel) défini par l'équation de récurrence :

$$u_{\text{filtré}}(t) = \alpha \cdot u_{\text{brut}}(t) + (1 - \alpha) \cdot u_{\text{filtré}}(t - 1) \quad (4)$$

Où $\alpha = \text{self.alpha_lissage}$ est le facteur de pondération compris entre 0 et 1.

- **Si $\alpha = 1.0$** : Le filtre est totalement inactif. La commande filtrée est strictement égale à la commande brute calculée par le PD. Le robot réagit instantanément à la moindre variation pixel ou LiDAR : son comportement visuel sera **saccadé, nerveux**, marqué par des secousses violentes au niveau des moteurs.
- **Si $\alpha = 0.01$** : Le lissage est excessivement lourd. Le robot conserve à 99% la mémoire de sa commande passée et n'intègre qu'à 1% la nouvelle consigne. Visuellement, les commandes seront extrêmement lentes à évoluer. Le robot aura une inertie logicielle critique, le rendant **incapable de tourner à temps** dans les virages, ce qui mènera à une collision directe par manque de réactivité.